

Homework

by

Christopher Arias

carias1@stevens.edu

October 11, 2024

© Christopher Arias
carias1@stevens.edu
ALL RIGHTS RESERVED

Homework

Christopher Arias
carias1@stevens.edu

This document provides the requirements and design details of the PROJECT. The following table (Table 1) should be updated by authors whenever major changes are made to the architecture design or new components are added. Add updates to the top of the table. Most recent changes to the document should be seen first and the oldest last.

Table 1: Document Update History

Date	Updates
9/19/2024	DDM: • Removed all unnecessary pages from template • Added new chapter for Homework Two (Chapter 4) • Inserted all necessary figures into the Homework Two chapter (Chapter 4)
10/10/24	DDM: • Inserted all of the answers for the Software Execution Model Homework (Chapter 5)

Table of Contents

1	Introduction	
	– <i>Christopher Arias</i>	1
2	Team	
	– <i>Christopher Arias</i>	2
3	Git Homework	
	– <i>Christopher Arias</i>	3
4	UML Class Modeling	
	– <i>Christopher Arias</i>	6
4.1	Exercise 2.1	6
4.2	Exercise 2.2	6
4.3	Exercise 2.3	7
4.4	Exercise 2.4	9
4.5	Code Related Figures and Images	10
5	Software Execution Models	
	– <i>Christopher Arias</i>	13
5.1	Exercise 5.1	13
5.2	Exercise 5.2	14
5.3	Exercise 5.3	15
	Bibliography	16

List of Tables

1	Document Update History	iii
---	-----------------------------------	-----

List of Figures

3.1	First test UML diagram from visual paradigm	4
3.2	Second test UML diagram from visual paradigm	5
3.3	Demonstrating completion of part 2 of this assignment	5
4.1	UML Diagram describing the undirected graph from Exercise 2.1	6
4.2	UML Diagram describing the directed graph from Exercise 2.2	6
4.3	Output of the modified ProductSale.py code	10
4.4	Updated UML diagram for code	12
5.1	Sequence Diagram describing how the search is conducted in Exercise 5.3	15

Chapter 1

Introduction

– *Christopher Arias*

This project is for completing the homework in SSW 345.

Chapter 2

Team

– *Christopher Arias*

Hello, I am Christopher Arias. I am a 4/4 computer science major with a minor in software engineering. I aspire to work for a big tech company like Google or Amazon.

Chapter 3

Git Homework

– *Christopher Arias*

[Link to repo](#)

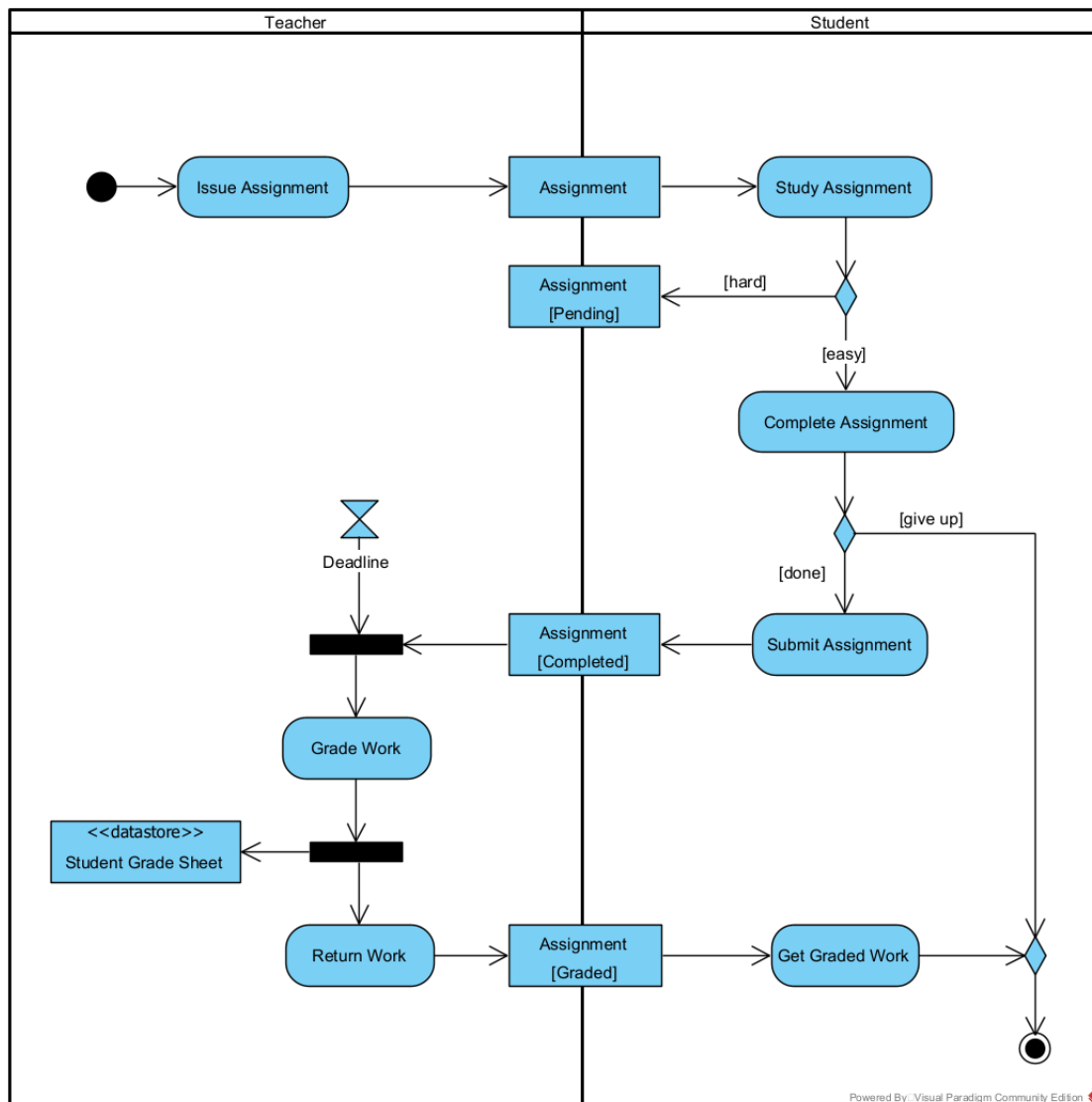


Figure 3.1: First test UML diagram from visual paradigm

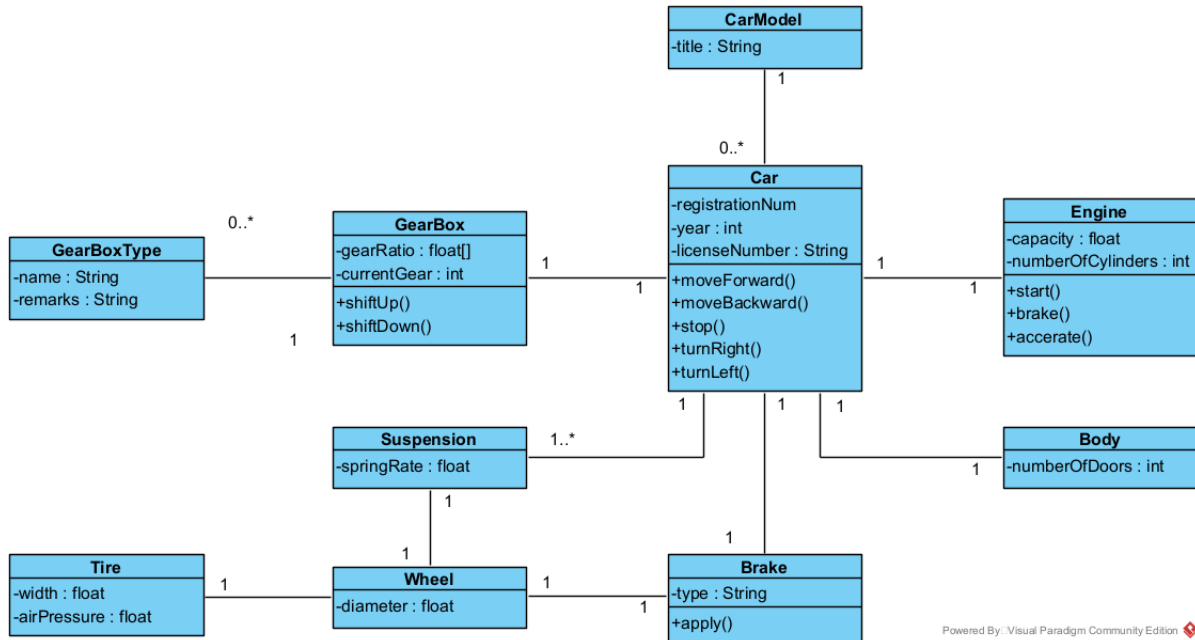


Figure 3.2: Second test UML diagram from visual paradigm

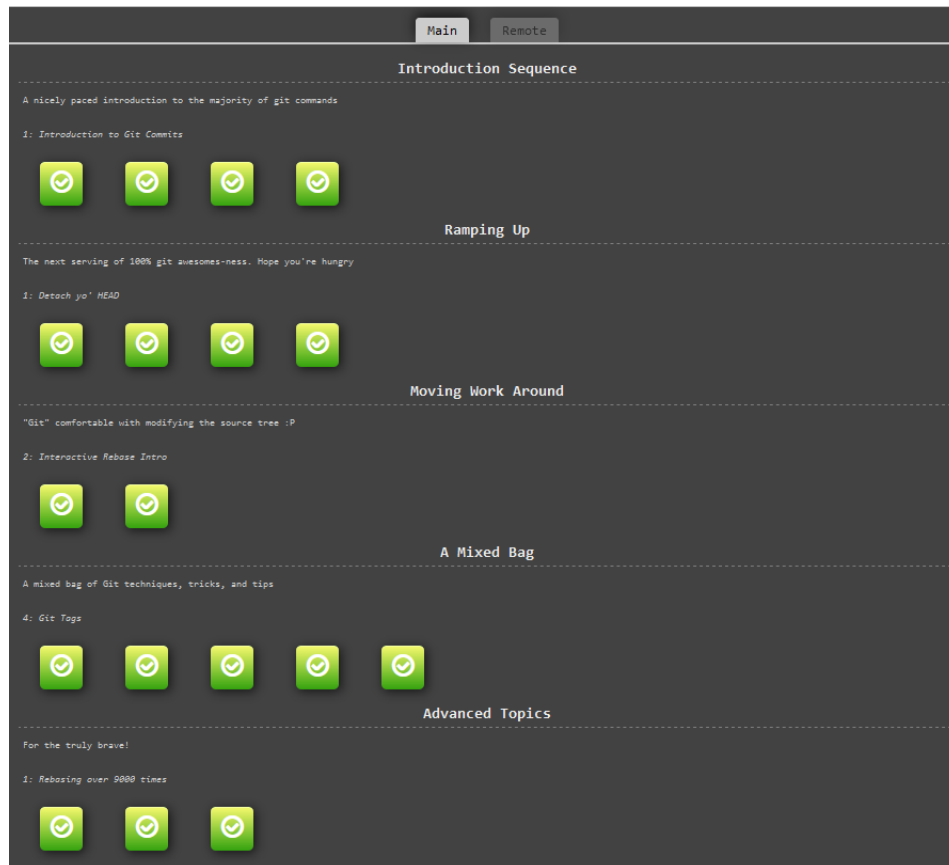


Figure 3.3: Demonstrating completion of part 2 of this assignment

Chapter 4

UML Class Modeling

– Christopher Arias

4.1 Exercise 2.1

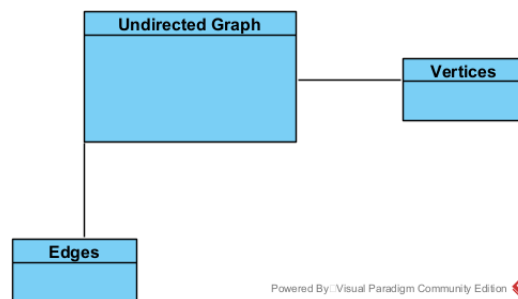


Figure 4.1: UML Diagram describing the undirected graph from Exercise 2.1

4.2 Exercise 2.2

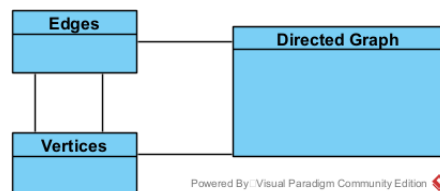


Figure 4.2: UML Diagram describing the directed graph from Exercise 2.2

4.3 Exercise 2.3

- Window to Canvas Association:

A single Window can be linked to multiple Canvas objects. A Window is responsible for displaying content, and it can host multiple Canvas objects. Each Canvas serves as an individual drawing surface within the Window.

- Canvas to Shape Association:

A Canvas can include zero or more Shape objects, with each Shape associated with only one Canvas. Canvas acts as a container for graphical elements like Shapes (such as Lines, ClosedShapes, etc.), which are drawn within the Canvas.

- Shape to Polygon, Line, ClosedShape, Ellipse Association:

The Shape class can be specialized into subtypes such as Line, Polygon, ClosedShape, and Ellipse. Shape serves as a generic class for specific graphical elements like Lines, Polygons, and Ellipses, which inherit Shape's properties and override its draw functionality.

- Canvas to ScrollingCanvas, TextWindow Association:

A Canvas can be specialized into either a ScrollingCanvas or a TextWindow. The Canvas can either represent a simple drawing area like a TextWindow or one with additional scrolling functionality like a ScrollingCanvas, allowing for varied content management.

- Panel to PanelItem Association:

A Panel can contain either zero or one PanelItem, and each PanelItem is linked to exactly one Panel. The Panel serves as a container for a single PanelItem, which represents interactive elements like buttons or choice items.

- PanelItem to Button, ChoiceItem, TextItem Association:

The PanelItem class can be specialized into different interactive types like Button, ChoiceItem, or TextItem. PanelItem generalizes elements that users can interact with, such as buttons for clicking, choice items for selection, and text items for input.

- ChoiceItem to ChoiceEntry Association:

A ChoiceItem contains a set of ChoiceEntry objects. Each ChoiceItem can be associated with multiple ChoiceEntries, representing the different selectable options within the ChoiceItem.

- PanelItem to Event Association:

A PanelItem is linked to one or more Event objects. PanelItems trigger Events based on user interactions, such as clicking a button or entering text, which result in specific actions being initiated.

- Event to PanelItem Association:

Each Event is associated with one specific PanelItem. Events are directly triggered by interactions with specific PanelItems, ensuring the right action is executed when a user interacts with a given PanelItem.

- Polygon to Point Association:

A Polygon is made up of multiple Point objects in a specific order. Polygons are defined by an ordered collection of Points, which represent the vertices of the polygon, forming its geometric structure.

4.4 Exercise 2.4

- Customer to MailingAddress Association:

A Customer can have several MailingAddresses, indicating a one-to-many relationship. The MailingAddress contains relevant details such as the address and phone number associated with the customer.

- Customer to CreditCardAccount Association:

A Customer can possess multiple CreditCardAccounts, but each account is tied to only one Customer. This relationship is represented by the "accountHolder" attribute, with each CreditCardAccount storing information such as maximum credit, current balance, and the statement date.

- CreditCardAccount to MailingAddress Association:

A CreditCardAccount is connected to exactly one MailingAddress. This association reflects the mailing address linked to the account for billing and correspondence.

- CreditCardAccount to Institution Association:

Each CreditCardAccount is related to one Institution. The Institution maintains details about the bank or organization responsible for managing the account, including its name, address, and phone number.

- CreditCardAccount to Statement Association:

A CreditCardAccount can be associated with multiple Statements. Each Statement provides detailed information such as the payment due date, finance charge, and minimum payment, and is linked to a single CreditCardAccount.

- Statement to Transaction Association:

A Statement may include multiple Transactions. Each Transaction contains specific details like the transaction date, explanation, and amount, and is associated with only one Statement.

- Transaction to CashAdvance, Interest, Purchase, Fee, Adjustment Association:

A Transaction can represent different types, including CashAdvance, Interest, Purchase, Fee, or Adjustment. Each transaction type refers to a specific kind of financial operation performed on the account.

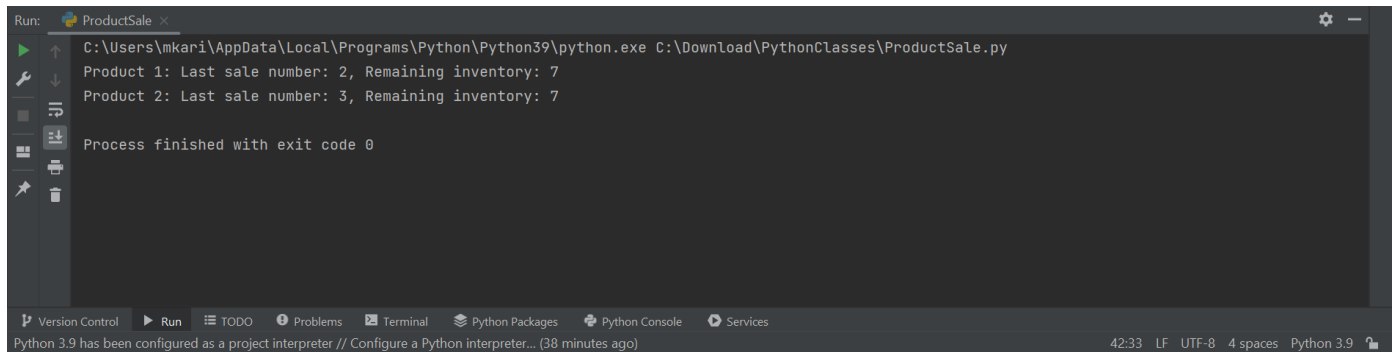
- Purchase to Merchant Association:

A Purchase transaction is linked to a Merchant. This relationship indicates that every purchase transaction may involve details about the Merchant from whom the purchase was made.

- Fee to FeeType Association:

A Fee transaction is related to a particular FeeType. The FeeType defines the nature of the fee applied in a transaction, such as late fees or service charges.

4.5 Code Related Figures and Images



The screenshot shows a terminal window titled "Run: ProductSale" with a dark background. The command executed is `C:\Users\mkari\AppData\Local\Programs\Python\Python39\python.exe C:\Download\PythonClasses\ProductSale.py`. The output consists of two lines: `Product 1: Last sale number: 2, Remaining inventory: 7` and `Product 2: Last sale number: 3, Remaining inventory: 7`. Below the output, it says `Process finished with exit code 0`. The IDE interface includes a sidebar with icons for Run, Debug, and other tools, and a bottom status bar showing "Python 3.9 has been configured as a project interpreter // Configure a Python interpreter... (38 minutes ago)" and "42:33 LF UTF-8 4 spaces Python 3.9".

```
Run: ProductSale
C:\Users\mkari\AppData\Local\Programs\Python\Python39\python.exe C:\Download\PythonClasses\ProductSale.py
Product 1: Last sale number: 2, Remaining inventory: 7
Product 2: Last sale number: 3, Remaining inventory: 7
Process finished with exit code 0
```

Figure 4.3: Output of the modified ProductSale.py code

ProductSale.py

```
1 # needed for forward reference of Sale in Product ,
2 # since Sale is not yet defined.
3 from __future__ import annotations
4 from typing import List
5
6 # forward reference used for class Sale
7 class Product:
8     __lastSale: Sale = None
9     __inventory: int = 0
10
11     def __init__(self, sale: Sale, inventory: int):
12         self.__lastSale = sale
13         self.__inventory = inventory
14
15     def setLastSale(self, lastSale: Sale):
16         self.__lastSale = lastSale
17
18     @property
19     def getLastSale(self) -> Sale:
20         return self.__lastSale
21
22     @property
23     def getInventory(self) -> int:
24         return self.__inventory
25
26     def reduceInventory(self, quantity: int):
27         if quantity > self.__inventory:
28             raise ValueError("Not enough inventory to fulfill sale.")
29         self.__inventory -= quantity
30
31     def __getitem__(self, item):
32         return self
33
34 # no forward reference needed since Product is defined
35 class Sale:
36     __saleTimes = 0
37     __productSold: List[Product] = None
38     __saleNumber: int = 0
39
40     def __init__(self, product: List[Product], quantities: List[int]):
41         Sale.__saleTimes += 1
42         self.__product = product
43         self.__saleNumber = Sale.__saleTimes
44
45         for index, product in enumerate(product):
46             if index < len(quantities):
47                 product.reduceInventory(quantities[index])
48                 product.setLastSale(self)
49
50     def setProductsSold(self, productSold: List[Product]):
51         self.__productSold = productSold
```

```

52
53     @property
54     def getSaleNumber(self) -> int:
55         return self.__saleNumber
56
57
58 productOne = Product(sale=None, inventory=10)
59 productTwo = Product(sale=None, inventory=15)
60
61 saleOne = Sale([productOne, productTwo], [2, 3])
62 saleTwo = Sale([productOne], [1])
63 saleThree = Sale([productTwo], [5])
64
65 print(f"Product 1: Last sale number: {productOne.getLastSale.getSaleNumber},
        Remaining inventory: {productOne.getInventory}")
66 print(f"Product 2: Last sale number: {productTwo.getLastSale.getSaleNumber},
        Remaining inventory: {productTwo.getInventory}")

```



Figure 4.4: Updated UML diagram for code

Chapter 5

Software Execution Models

– Christopher Arias

5.1 Exercise 5.1

Times for each unit:

WorkUnit:

$$(3 \times 400 \times .0001) + (2 \times 0 \times .02) + (1 \times 0 \times .01) = 0.12$$

DBUnit:

$$(3 \times 1500 \times .0001) + (2 \times 150 \times .02) + (1 \times 2 \times .01) = 6.47$$

MsgUnit:

$$(3 \times 10 \times .0001) + (2 \times 0 \times .02) + (1 \times 1 \times .01) = 0.013$$

Times for each state:

$$P_0 : (775 \times .0001) + (14 \times .02) + (244 \times .01) = 2.7975$$

$$P_1 : 4(\text{WorkUnit}) + 0(\text{DBUnit}) + 1(\text{MsgUnit}) = 4(.12) + 0(6.47) + 1(.013) = .493$$

$$P_3 : 0(\text{WorkUnit}) + 0(\text{DBUnit}) + 1(\text{MsgUnit}) = 0(.12) + 0(6.47) + 1(.013) = .013$$

$$P_5 : 1(\text{WorkUnit}) + 2(\text{DBUnit}) + 3(\text{MsgUnit}) = 1(.12) + 2(6.47) + 3(.013) = 13.099$$

$$P_6 : 0(\text{WorkUnit}) + 0(\text{DBUnit}) + 1(\text{MsgUnit}) = 0(.12) + 0(6.47) + 1(.013) = .013$$

$$P_7 : 0(\text{WorkUnit}) + 1(\text{DBUnit}) + 0(\text{MsgUnit}) = 0(.12) + 1(6.47) + 0(.013) = 6.47$$

$$P_{10} : 10(\text{WorkUnit}) + 1(\text{DBUnit}) + 1(\text{MsgUnit}) = 10(.12) + 1(6.47) + 1(.013) = 7.683$$

$$P_{11} : 0(\text{WorkUnit}) + 0(\text{DBUnit}) + 1(\text{MsgUnit}) = 0(.12) + 0(6.47) + 1(.013) = .013$$

$$P_{12} : 3(\text{WorkUnit}) + 2(\text{DBUnit}) + 4(\text{MsgUnit}) = 3(.12) + 2(6.47) + 4(.013) = 13.352$$

$$P_{13} : 8(\text{WorkUnit}) + 3(\text{DBUnit}) + 1(\text{MsgUnit}) = 8(.12) + 3(6.47) + 1(.013) = 20.383$$

$$\text{Initial Path: } (P_0 \rightarrow P_1 \rightarrow P_0): P_0 + P_1 + P_0 = 6.088$$

$$\text{Beginning of Loop: } (P_3 \rightarrow P_0): P_3 + P_0 = 2.8105$$

$$\text{Loop w/ P5 path: } (4(\text{BeginningOfLoop} \rightarrow P_5 \rightarrow \max(P_6, P_7)): 4(\text{BeginningOfLoop} + P_5 + P_7) = 89.518$$

$$\text{Loop w/ P0 Path: } (4(\text{BeginningOfLoop} \rightarrow P_0): 4(\text{BeginningOfLoop} + P_0) = 22.432$$

$$\text{Loop w/ P1 Path: } (4(\text{BeginningOfLoop} \rightarrow P_1 \rightarrow \min(P_{10}, P_{11}, P_{12})): 4(\text{BeginningOfLoop} + P_1 + P_{11}) = 13.266$$

$$\text{Loop w/ P13 Path: } (4(\text{BeginningOfLoop} \rightarrow P_{13}): 4(\text{BeginningOfLoop} + P_{13}) = 92.774$$

Total Path following P5 Path: (Initial Path + Loop w/ P5 path): $6.088 + 89.518 = 95.606$
 Total Path following P0 Path: (Initial Path + Loop w/ P0 path): $6.088 + 22.432 = 28.52$
 Total Path following P1 Path: (Initial Path + Loop w/ P1 path): $6.088 + 13.266 = 19.354$
 Total Path following P13 Path: (Initial Path + Loop w/ P13 path): $6.088 + 92.774 = 98.862$

Best Case time: P1 Path (19.354 seconds)

Worst Case time: P13 Path (98.862 seconds)

Avg Case time: Initial Path + 4(Beginning of Loop + (Probability of P5 path)*(P5 Path) + (Probability of P0 path)*(P0 Path) + (Probability of P1 path)*(P1 Path) + (Probability of P13 path)*(P13 Path))
 $= 6.088 + 4(2.8105 + (.1)(19.569) + (.2)(2.7975) + (.3)(.506) + (.4)(20.383)) = 60.6156$

5.2 Exercise 5.2

Times for each unit:

WorkUnit:

$$(3 \times 25 \times .0001) + (2 \times 0 \times .02) + (1 \times 0 \times .1) = .0075$$

DBUnit:

$$(3 \times 400 \times .0001) + (2 \times 4 \times .02) + (1 \times 2 \times .1) = 0.48$$

MsgUnit:

$$(3 \times 25 \times .0001) + (2 \times 0 \times .02) + (1 \times 1 \times .1) = 0.1075$$

Times for each state:

$$\text{initialize: } 2(\text{WorkUnit}) + 1(\text{DBUnit}) + 1(\text{MsgUnit}) = .6025$$

$$\text{update: } 1(\text{WorkUnit}) + 0(\text{DBUnit}) + 0(\text{MsgUnit}) = .0075$$

$$\text{query: } .0075$$

$$\text{write: } .0075$$

$$\text{updateMirrorNY: } 5(\text{WorkUnit}) + 2(\text{DBUnit}) + 2(\text{MsgUnit}) = 1.2125$$

$$\text{updateMirrorLA: } 4(\text{WorkUnit}) + 2(\text{DBUnit}) + 2(\text{MsgUnit}) = 1.205$$

$$\text{queryMirrorNY: } 3(\text{WorkUnit}) + 1(\text{DBUnit}) + 2(\text{MsgUnit}) = .7175$$

$$\text{queryMirrorLA: } 4(\text{WorkUnit}) + 1(\text{DBUnit}) + 2(\text{MsgUnit}) = .725$$

$$\text{writeMirrorNY: } 3(\text{WorkUnit}) + 1(\text{DBUnit}) + 2(\text{MsgUnit}) = .7175$$

$$\text{writeMirrorLA: } 4(\text{WorkUnit}) + 1(\text{DBUnit}) + 2(\text{MsgUnit}) = .725$$

$$\text{Total Path ending at updateMirrorNY: Initialize + Update + updateMirrorNY: } 2(.6025 + .0075 + 1.2125) = 3.645$$

$$\text{Total Path ending at updateMirrorLA: Initialize + Update + updateMirrorLA: } 2(.6025 + .0075 + 1.205) = 3.63$$

$$\text{Total path ending at min(queryMirrorNY, queryMirrorLA): Initialize + Query + queryMirrorNY: } 2(.6025 + .0075 + .7175) = 2.655$$

$$\text{TotalPath ending at max(writeMirrorNY, writeMirrorLA): Initialize + Write + writeMirrorLA: } 2(.6025 + .0075 + .725) = 2.67$$

Best Case time: Total path ending at min(queryMirrorNY, queryMirrorLA) (2.655 seconds)

Worst Case time: Total path ending at updateMirrorNY (3.645 seconds)

Avg Case time:

$$\begin{aligned}
 &2(\text{Initialize} + (\\
 &(\text{ProbUpdate})(\text{Update} + (\text{ProbUpdateMirrorNY} * \text{updateMirrorNY}) + (\text{ProbUpdateMirrorLA} * \text{updateMirrorLA})) \\
 &+ ((\text{ProbQuery})(\text{Query} + \text{queryMirrorNY})) \\
 &+ ((\text{ProbWrite})(\text{Write} + \text{writeMirrorLA})) \\
 &)) \\
 &= 3.1491 \text{ seconds}
 \end{aligned}$$

5.3 Exercise 5.3

1.

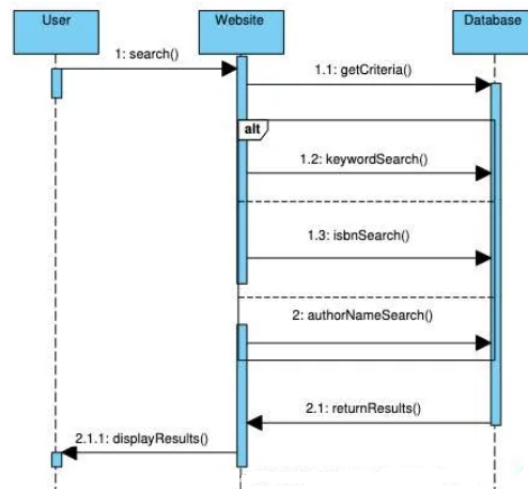


Figure 5.1: Sequence Diagram describing how the search is conducted in Exercise 5.3

Bibliography

Index

Chapter

GitHomework, [3](#)

Introduction, [1](#)

SoftwareExecutionModels, [13](#)

Team, [2](#)

UMLClassModeling, [6](#)

githomework, [3](#)

introduction, [1](#)

Software Execution Models, [13](#)

team, [2](#)

UML Class Modeling, [6](#)